

SmartCareer — UAT Test Scripts: Security, Concurrency & Edge Cases

Document Code: UAT-DOC-04

Focus Area: SECURITY, RBAC, CONCURRENCY, RESILIENCY & EDGE CASES

Status: READY FOR EXECUTION

Standards Reference: OWASP Top 10 API Security Risks, ISO/IEC 25010 Quality Standards

Index Reference: [00_UAT_MASTER_INDEX.md](#)

1. วัตถุประสงค์และความสำคัญ (Security Testing Objectives)

การทดสอบในเอกสารฉบับนี้มีเป้าหมายเพื่อพิสูจน์ความแข็งแกร่งของระบบ SmartCareer ในแง่:

- การป้องกันการเข้าถึงข้อมูลโดยมิชอบ (Insecure Direct Object References - IDOR)
- การแบ่งแยกขอบเขตข้อมูลอย่างเคร่งครัด (Multi-Tenant Isolation)
- ความสมบูรณ์และการป้องกันการปลอมแปลง Token (JWT Integrity & Session Expiry)
- ความปลอดภัยของระบบรันโค้ด Sandbox (Judge0 Code Containment & Abuse Prevention)
- การรับมือกับการส่งคำขอพร้อมกันและ Race Conditions (Double-Submit & Concurrency)
- ความยืดหยุ่นเมื่อบริการภายนอกล่ม (Fault-Tolerance & External Outage Resilience)

2. รายการกรณีทดสอบความปลอดภัย (Security & Concurrency Scenarios)

```
graph TD
  TD[Security & Concurrency Testing] --> Access[1. Access Control & IDOR]
  TD --> AuthSec[2. Token Integrity & CSRF]
  TD --> Sandbox[3. Judge0 Sandbox Containment]
  TD --> Concurrency[4. Concurrency & Race Conditions]
  TD --> Resilience[5. Resilience & Outage Recovery]
```

[TC-SEC-01] Cross-User IDOR Protection (Candidate Data Isolation)

- Scenario ID: TC-SEC-01
- Security Category: OWASP API1:2023 Broken Object Level Authorization (IDOR)
- Target Endpoints: GET /api/candidate/profile , GET /api/candidate/applications
- Test Setup: มี Candidate A และ Candidate B อยู่ในระบบ

ขั้นตอนการทดสอบ (Step-by-Step):

1. ล็อกอินเป็น Candidate A และคัดลอก Bearer JWT Token ของ Candidate A
2. ใช้ Postman หรือ `curl` ส่ง Request ไปยัง `GET /api/candidate/profile` โดยแนบ Token ของ Candidate A
 - o คาดหวัง: ได้รับข้อมูลเฉพาะของ Candidate A เท่านั้น
3. สังเกตว่า API ไม่มี Parameter `userId` ให้สลับใน URL แต่ดึงจาก `req.user.id` ใน Token
4. ทดสอบส่ง Request ไปยัง `GET /api/assessments/my-attempts`
 - o คาดหวัง: ได้รับเฉพาะประวัติการสอบของ Candidate A ไม่ปะปนกับ Candidate B
5. พยายามส่ง Request ไปยัง `GET /api/assessments/attempts/{attempt_id_of_candidate_B}/review` โดยใช้ Token ของ Candidate A

ผลลัพธ์ที่คาดหวัง (Expected Results):

- Backend ส่ง HTTP 403 Forbidden หรือ HTTP 404 Not Found ไม่อนุญาตให้ Candidate A ดูผลสอบของ Candidate B
- **Acceptance Verdict:** `[REAL INTEGRATION PASS / FAIL]`

[TC-SEC-02] Cross-Company Isolation Guard (Applicant Privacy)

- **Scenario ID:** `TC-SEC-02`
- **Security Category:** Multi-Tenant Isolation & Privacy Boundary
- **Target Endpoint:** `GET /api/company/applications` และ `POST /api/evaluations/:applicationId`
- **Test Setup:**
 - o Company A (`TechCorp`) มีประกาศงาน Job 1 และมีผู้สมัคร App 1
 - o Company B (`Siam Cloud`) มีประกาศงาน Job 2 และมีผู้สมัคร App 2

ขั้นตอนการทดสอบ (Step-by-Step):

1. ล็อกอินเป็น Company B และคัดลอก Bearer Token ของ Company B
2. ใช้ Postman พยายามดึงข้อมูลใบสมัคร App 1 ของ Company A:

```
curl -X GET "http://localhost:4000/api/company/applications?jobId={job1_id}" \
-H "Authorization: Bearer {token_of_company_B}"
```

3. ใช้ Token ของ Company B พยายามส่งแบบประเมินไปยัง App 1 ของ Company A:

```
curl -X POST "http://localhost:4000/api/evaluations/{app1_id}" \
-H "Authorization: Bearer {token_of_company_B}" \
-H "Content-Type: application/json" \
-d '{"technicalScore": 1, "overallFeedback": "Malicious evaluation attempt"}'
```

ผลลัพธ์ที่คาดหวัง (Expected Results):

- ในขั้นตอนที่ 2: Backend ไม่แสดงรายชื่อผู้สมัครของ Company A

- ในขั้นตอนที่ 3: Backend ปฏิเสธด้วย HTTP 403 Forbidden ("Cannot evaluate application for another company")
 - **Acceptance Verdict:** [REAL INTEGRATION PASS / FAIL]
-

[TC-SEC-03] Admin API Direct Access Guard without Bearer Token

- **Scenario ID:** TC-SEC-03
- **Security Category:** OWASP API5:2023 Broken Function Level Authorization
- **Target Endpoints:** GET /api/admin/dashboard , PUT /api/admin/verifications/:id/review

ขั้นตอนการทดสอบ (Step-by-Step):

1. ส่ง Request ไปยัง GET /api/admin/dashboard โดย **ไม่แนบ** Authorization Header
 - คาดหวัง: HTTP 401 Unauthorized
2. ล็อกอินเป็น Candidate หรือ Company และนำ Token ดังกล่าวส่ง Request ไปยัง GET /api/admin/dashboard
 - คาดหวัง: HTTP 403 Forbidden (Forbidden resource)
3. ใช้ Token ของ Candidate พยายามยิง Request อนุมัติบริษัทตนเอง:
PUT /api/admin/verifications/{id}/review พร้อม Body {"action": "APPROVE"}
 - คาดหวัง: HTTP 403 Forbidden ทันที

ผลลัพธ์ที่คาดหวัง (Expected Results):

- RolesGuard และ JwtAuthGuard ทำงานสกัดกั้นการเข้าถึงระดับ Controller ได้ 100%
 - **Acceptance Verdict:** [REAL INTEGRATION PASS / FAIL]
-

[TC-SEC-04] Token Integrity: Malformed, Expired or Forged JWT

- **Scenario ID:** TC-SEC-04
- **Security Category:** Cryptographic Integrity & Session Management
- **Target Endpoint:** GET /api/auth/me

ขั้นตอนการทดสอบ (Step-by-Step):

1. **ทดสอบ Token ปลอม (Forged Signature):** นำ JWT Token ที่ถูกต้องมาแก้ Payload ในส่วน Signature แม้เพียง 1 ตัวอักษร แล้วส่ง Request
 - คาดหวัง: HTTP 401 Unauthorized (invalid signature)
2. **ทดสอบ Token ผิดรูปแบบ (Malformed String):** ส่ง Header Authorization: Bearer not-a-valid-token-string
 - คาดหวัง: HTTP 401 Unauthorized (jwt malformed)
3. **ทดสอบ Token หมดอายุ (Expired Token):** ส่ง Token ที่ตั้งเวลา exp ในอดีต
 - คาดหวัง: HTTP 401 Unauthorized (jwt expired)

ผลลัพธ์ที่คาดหวัง (Expected Results):

- NestJS Passport JWT Strategy ตรวจสอบและปฏิเสธ Token ที่ไม่สมบูรณ์ทุกรูปแบบ
 - **Acceptance Verdict:** [REAL INTEGRATION PASS / FAIL]
-

[TC-SEC-05] OAuth State Parameter & CSRF Tampering Guard

- **Scenario ID:** TC-SEC-05
- **Security Category:** Cross-Site Request Forgery (CSRF) in OAuth Flows
- **Target Endpoint:** GET /api/auth/github/callback

ขั้นตอนการทดสอบ (Step-by-Step):

1. ส่ง Request ไปยัง GET /api/auth/github/callback?code=fake_code&state=tampered_state
2. ตรวจสอบว่าระบบสามารถถอดรหัส State หรือตรวจจับค่า State ที่ไม่ถูกต้องและดีดกลับหน้า Login พร้อม Error หรือไม่

ผลลัพธ์ที่คาดหวัง (Expected Results):

- ระบบจัดการ Error อย่างปลอดภัย ไม่เกิด Unhandled Crash บน Server
 - **Acceptance Verdict:** [PASS / FAIL]
-

[TC-SEC-06] Judge0 Sandbox Abuse & Isolation Containment

- **Scenario ID:** TC-SEC-06
- **Security Category:** Remote Code Execution (RCE) Sandbox Containment
- **Target Endpoint:** POST /api/assessments/:id/run-code

ขั้นตอนการทดสอบ (Step-by-Step):

1. ทดสอบ Fork Bomb / Resource Exhaustion:

ส่งโค้ดที่พยายามระเบิด Process:

```
function solution() {  
  const cluster = require('child_process');  
  while(true) { cluster.fork(); }  
}
```

2. ทดสอบการเข้าถึง Filesystem Host Server:

ส่งโค้ดที่พยายามอ่านไฟล์ Host:

```
function solution() {  
  const fs = require('fs');  
  return fs.readFileSync('/etc/passwd', 'utf8');  
}
```

3. ทดสอบ Network Egress:

ส่งโค้ดที่พยายามยิง HTTP ออกไปยังภายนอก

ผลลัพธ์ที่คาดหวัง (Expected Results):

- โค้ดทั้งหมดต้องถูกส่งไปรันใน **RapidAPI Judge0 Docker Container** ที่ถูกจำกัดสิทธิ์ (Isolated Sandbox)
- โค้ดของผู้สมัคร **ต้องไม่มีวันถูก execute** ภายใน **NestJS Backend Node.js process**
- หากพยายามรันคำสั่งอันตราย Judge0 ตอบกลับ `RUNTIME_ERROR` หรือ `COMPILE_ERROR` โดย Server หลักของ SmartCareer ไม่ได้รับผลกระทบใดๆ
- **Acceptance Verdict:** [REAL INTEGRATION PASS / FAIL]

[TC-SEC-07] Rate Limiting Enforcement (Judge0 3s Throttle)

- **Scenario ID:** TC-SEC-07
- **Security Category:** Denial of Service (DoS) & API Abuse Prevention
- **Target Endpoint:** POST /api/assessments/:id/run-code

ขั้นตอนการทดสอบ (Step-by-Step):

1. เขียนสคริปต์ยิงคำขอ `run-code` ติดต่อกัน 2 ครั้งภายในเวลา 500 มิลลิวินาที
2. สังเกต Response ของ Request ที่สอง

ผลลัพธ์ที่คาดหวัง (Expected Results):

- Request ที่สองได้รับ HTTP 429 Too Many Requests:
"Rate limit exceeded. Please wait 3s before running tests again."
- ช่วยป้องกันการสแปมเรียก Judge0 API สิ้นเปลืองโควตา
- **Acceptance Verdict:** [REAL INTEGRATION PASS / FAIL]

[TC-SEC-08] Payload Size Limit Enforcement (64KB Code Cap)

- **Scenario ID:** TC-SEC-08
- **Security Category:** Payload Size Buffer Protection
- **Target Endpoint:** POST /api/assessments/:id/submit-coding

ขั้นตอนการทดสอบ (Step-by-Step):

1. สร้างสตริงโค้ดขนาดใหญ่เกิน 64 KB (เช่น ข้อความซ้ำๆ 70,000 ตัวอักษร)
2. ส่ง Request ไปยัง `submitCodingSolution`

ผลลัพธ์ที่คาดหวัง (Expected Results):

- Backend ปฏิเสธทันทีด้วย HTTP 400 Bad Request:
"Source code exceeds 64KB size limit"

- **Acceptance Verdict:** [PASS / FAIL]

[TC-SEC-09] Concurrency & Double-Submit Protection (Race Condition)

- **Scenario ID:** TC-SEC-09
- **Security Category:** Race Condition & Transactional Locking
- **Target Endpoints:** POST /api/applications/:jobId/apply , POST /api/assessments/:id/submit-coding

ขั้นตอนการทดสอบ (Step-by-Step):

1. จำลองการคลิกปุ่ม "ยืนยันการสมัครงาน" หรือ "ส่งข้อสอบ" รัวๆ 2 ครั้งพร้อมกันในเสี้ยววินาที (Concurrent Requests)
2. ตรวจสอบว่าในฐานข้อมูลเกิด Record ซ้ำหรือไม่

ผลลัพธ์ที่คาดหวัง (Expected Results):

- Request แรกสำเร็จ (HTTP 201)
- Request ที่สองถูกสกัดกั้นด้วย Database Unique Constraint หรือ State Machine (Attempt status is already SUBMITTING/COMPLETED)
- ไม่เกิดข้อมูลซ้ำซ้อนในฐานข้อมูล
- **Acceptance Verdict:** [REAL INTEGRATION PASS / FAIL]

[TC-SEC-10] Anti-Cheat Integrity Logging (Tab Blur / Window Switch)

- **Scenario ID:** TC-SEC-10
- **Security Category:** Exam Proctoring & Integrity Verification
- **Target Route:** http://localhost:3000/assessments/[id]

ขั้นตอนการทดสอบ (Step-by-Step):

1. เข้าทำข้อสอบ Coding Assessment
2. ระหว่างทำข้อสอบ ทำการสลับแท็บไปหน้าอื่น หรือกด Alt+Tab สลับโปรแกรม
3. สลับกลับมายังหน้าทำข้อสอบ
4. สังเกตแถบแบนเนอร์แจ้งเตือนสีเหลืองด้านบน
5. ตรวจสอบข้อมูลในตาราง assessment_attempts ในฟิลด์ integrityEvents

ผลลัพธ์ที่คาดหวัง (Expected Results):

- หน้าเว็บแสดงข้อความแจ้งเตือนตรวจพบการสลับหน้าต่าง
- ส่ง Event บันทึกลงฐานข้อมูล: {"type": "TAB_BLUR", "timestamp": "...", "details": "Candidate blurred active window"}
- บริษัทและ Tech Lead สามารถตรวจสอบความซื่อสัตย์ในการสอบได้
- **Acceptance Verdict:** [PASS / FAIL]

[TC-SEC-11] Closed Job Application Guard (Expired / Inactive Job)

- **Scenario ID:** TC-SEC-11
- **Security Category:** Business Logic Integrity
- **Target Endpoint:** POST /api/applications/:jobId/apply

ขั้นตอนการทดสอบ (Step-by-Step):

1. ค้นหาตำแหน่งงานที่ `isActive = false` หรือ `expiresAt < now()`
2. พยายามส่ง Request ยื่นใบสมัครงานตำแหน่งดังกล่าวจริงๆ ผ่าน API

ผลลัพธ์ที่คาดหวัง (Expected Results):

- Backend ปฏิเสธด้วย HTTP 400 Bad Request:
"ตำแหน่งงานนี้ปิดรับสมัครแล้ว (This job application has closed)"
- **Acceptance Verdict:** [PASS / FAIL]

[TC-SEC-12] GitHub Account Lock Policy (Anti-Account Hijacking)

- **Scenario ID:** TC-SEC-12
- **Security Category:** Identity Integrity Protection
- **Target Endpoint:** PUT /api/candidate/profile และ POST /api/github/sync

ขั้นตอนการทดสอบ (Step-by-Step):

1. ล็อกอินเป็น Candidate ที่มีบัญชี GitHub เชื่อมโยงอยู่แล้ว (เช่น @natdanai-dev)
2. พยายามเรียก POST /api/github/sync โดยระบุ Body {"username": "other-developer"}

ผลลัพธ์ที่คาดหวัง (Expected Results):

- Backend ส่ง HTTP 403 Forbidden:
"You can only sync your verified GitHub account: @natdanai-dev. Linked accounts are permanently locked."
- ป้องกันการที่ Candidate นำผลงาน GitHub ของผู้อื่นมาสวมรอยเพิ่มคะแนนเรดาร์ตนเอง
- **Acceptance Verdict:** [PASS / FAIL]

[TC-SEC-13] External API Outage Resilience (Graceful Degradation)

- **Scenario ID:** TC-SEC-13
- **Security Category:** Availability & Fault-Tolerance

ขั้นตอนการทดสอบ (Step-by-Step):

1. ตัดการเชื่อมต่อของ RapidAPI Judge0 หรือ Google Gemini API ชั่วคราว (เช่น เปลี่ยน API Key เป็นค่าไม่ถูกต้องใน `.env`)
2. ทำการกดส่งข้อสอบ หรือกดรัน Ingestion
3. สังเกตการจัดการ Error ของระบบ

ผลลัพธ์ที่คาดหวัง (Expected Results):

- ข้อสอบ Open-Ended ที่เรียก Gemini ไม่แครช Server แต่ปรับสถานะเป็น `EVALUATION_PENDING` พร้อมแจ้งเตือนรอการประเมิน
 - การดึงงาน Ingestion ที่เชื่อมต่อภายนอกไม่ผ่าน จะบันทึกสถานะ `FAILED` หรือ `PARTIAL_SUCCESS` ใน `ingestion_logs` โดยไม่กระทบกับงานเดิมในระบบ
 - **Acceptance Verdict:** `[PASS / FAIL]`
-

[TC-SEC-14] Scheduler Duplicate Execution Lock & Concurrency

- **Scenario ID:** `TC-SEC-14`
- **Security Category:** Scheduled Tasks Concurrency Control

ขั้นตอนการทดสอบ (Step-by-Step):

1. ตรวจสอบการทำงานของ `@Cron(CronExpression.EVERY_DAY_AT_MIDNIGHT)` ใน `scheduler.service.ts`
2. ทดสอบจำลองการสั่งรัน Job Sync พร้อมๆ กันจากหลาย Background Threads

ผลลัพธ์ที่คาดหวัง (Expected Results):

- ระบบมี Deduplication Engine ตรวจสอบความซ้ำซ้อนของงานจาก `externalId` และ URL
- ไม่เกิดปัญหางานซ้ำ (Duplicate Jobs) ในฐานข้อมูล
- **Acceptance Verdict:** `[PASS / FAIL]`